Exact counterexample certificates to the shifted lonely runner conjecture for $18 \le n \le 38$

Xavier Callens (SocrateAI Lab)

Preprint v1, 2026-10-31

doi:10.5281/zenodo.23842167 (not peer reviewed)

Zenodo DOI to be assigned on release (not peer reviewed)

AI-assistance disclosure and audit status. This preprint, its search and checking code, its Lean code and its claim ledger were produced by AI agents (Claude, Anthropic) under the author's direction; the author is responsible for the content. **This version was independently audited by a different AI agent instance (`AUDIT.md`, ACCEPT_WITH_CORRECTIONS), not by a human.** The audit's eight corrections are applied in this text (Section 8). No human audit exists, and the repository's statement lock and `lean_runner` gate for the full set of Lean files are still owed (Section 10); the Lean tier below is therefore *Tier A pending a statement lock*.

Abstract

The *shifted* lonely runner conjecture (Blanco–Criado–Santos, arXiv:2603.24784, Conjecture 1.1(ii)) says: for distinct positive integer velocities $v_1, \dots, v_n$ and any real shifts $s_1, \dots, s_n$ there is a time t with $\operatorname{dist}(s_i + v_i t, \mathbb{Z}) \ge \frac{1}{n+1}$ for all i. It is known to fail for $5 \le n \le 17$ (Blanco–Criado–Santos) and for every $n \ge 95$ (Poliakova, arXiv:2609.23952); Zhang (arXiv:2608.13599) asks about the range $18 \le n \le 94$. **We give explicit exact certificates that it fails for every n with $18 \le n \le 38$:** distinct positive integers v_i and rational shifts $s_i = a_i/D$ for which the n open arcs of radius $\frac{1}{n+1}$ cover the circle. For $n \le 28$ the velocities are $\{1, \dots, n\}$; for $29 \le n \le 38$ they are sets with smooth numbers up to 76. The exact value of $\gamma(v; s) = \max_t \min_i \|s_i + v_i t\|$ is $< \frac{1}{n+1}$ in every case, with $1/\gamma - (n+1) \in [0.0961, 0.1098]$, hence $E_n \ge 0.096$ in Poliakova's notation for $n = 18, \dots, 38$. Each certificate was accepted by two checkers of the producer (different criteria and languages, *same author*) and by an independent auditor's third checker (two exact methods), and the exact covering statement was kernel-checked in Lean by the auditor for all 34 certificates $n = 5, \dots, 38$ (Tier [A] *pending a statement lock*); we re-ran the Lean gates for $n = 18$ and $n = 38$. **For $39 \le n \le 94$ no certificate was found within a 30 core-hour budget and the question stays open.** Together with the literature, the conjecture is known to fail for $5 \le n \le 38$ and $n \ge 95$. These are upper bounds $\Gamma(n) < \frac{1}{n+1}$ only. **Nothing is said about the unshifted lonely runner conjecture, and the statement "the shifted conjecture fails for all $n \ge 5$" is not supported by this work.**

Markers. [A]: kernel-checked Lean, axioms within {`propext`, `Classical.choice`, `Quot.sound`}, controls caught. [L]: a cited source, taken as stated. [C]: executed code with an argument written down but not machine-checked. [X]: floating-point search output or a recorded absence, supporting nothing by itself. Identifiers `SLR-·-nnnn` refer to `ledger/ledger.json`.

1 Setting and the definition as pinned

The definition was pinned by the auditor from the sources themselves (read on alphaXiv), not taken from the producer (`AUDIT.md` §1):

- **BCS** [1] (arXiv:2603.24784 v4, 24 Oct 2026), p. 2, Conjecture 1.1(ii): for non-zero real $v_1, \dots, v_n$ with all $|v_i|$ different and every $s_1, \dots, s_n \in \mathbb{R}$ there is $t \in \mathbb{R}$ with $\operatorname{dist}(s_i + v_i t, \mathbb{Z}) \geq \frac{1}{n+1}$ for all i. Def. 1.3: $\gamma(v; s) = \sup_t \min_i \operatorname{dist}(s_i + v_i t, \mathbb{Z})$ and $\gamma_{\mathrm{sh}}(v) = \min_s \gamma(v; s)$. Integer v is no loss of generality (p. 3). With repeated velocities the conjecture fails trivially (v_i all equal, $s_i = i/n$), so distinctness is essential.
- **Zhang** [2] (arXiv:2608.13599 v3), App. A: distinct positive integers, $s \in [0, 1)^n$, $\gamma = \max_t \min_i \|s_i + v_i t\|$, $\Gamma(n) = \min_{|v|=n} \gamma_{\min}(v)$; Question 8.5 asks whether $\Gamma(n) < \frac{1}{n+1}$ for every $n \geq 5$; p. 19: "open only for $18 \leq n \leq 94$".
- **Poliakova** [3] (arXiv:2609.23952 v1), §1: distinct positive integer velocities, shifts in $\mathbb{R}/\mathbb{Z}$, $\frac{1}{n+1+E_n} = \inf_{v,s} \max_t \min_i \|s_i + v_i t\|$; Theorem 2 ($E_n \geq 1$) covers only $n \geq 95$; p. 3: finding counterexamples for the remaining $18 \leq n \leq 94$ "would establish that SLR is false for all $n \geq 5$".

Consequences used in this paper (all [L], `SLR-L-0001`):

1. A counterexample of size n is n runners with pairwise distinct positive integer velocities, real shifts and $\gamma(v; s) < \frac{1}{n+1}$.
2. For integer v the function $t \mapsto \min_i \|s_i + v_i t\|$ is 1-periodic, so $t \in \mathbb{R}/\mathbb{Z}$. Its maximum is attained, so $\gamma < \delta$ iff for every t some i has $\|s_i + v_i t\| < \delta$ iff the *open* arcs $\{t : \|s_i + v_i t\| < \delta\}$ cover the circle. Tangency (a single uncovered point) is therefore *not* a cover. This is the criterion all checkers use with $\delta = \frac{1}{n+1}$.
3. Shifts matter mod 1 and rational shifts are a special case of real ones; positive velocities suffice.
4. A certificate gives upper bounds only: $\gamma_{\mathrm{sh}}(v) \leq \gamma(v; s) < \frac{1}{n+1}$, hence $\Gamma(n) < \frac{1}{n+1}$ and $E_n > 0$. With the exact value, $E_n \geq 1/\gamma(v; s) - (n + 1)$.

2 Result

Certificates. For every n with $5 \leq n \leq 38$ the file `data/certificates/cert_nNNN.json` holds v (distinct positive integers, $\#v = n$), integers $a_1, \dots, a_n$ and D, with $s_i = a_i/D$ and $\delta = \frac{1}{n+1}$. The 13 cases $n = 5, \dots, 17$ are known (BCS) and serve as positive controls with our own shifts; the 21 cases $n = 18, \dots, 38$ are the result. The table gives, for these, the exact $\gamma(v; s)$ computed by the auditor's checker (method B, exact integer arithmetic; re-run by us, `data/logs/auditor_checker_rerun.jsonl`) and the resulting margin.

n	velocities	$\max v$	D	$\gamma(v; s)$ (exact)	$1/\gamma - (n + 1)$
18	$\{1, \dots, 18\}$	18	5777	2117/40439	0.1020
19	$\{1, \dots, 19\}$	19	6400	7323/147200	0.1011
20	$\{1, \dots, 20\}$	20	7057	1003/21171	0.1077
21	$\{1, \dots, 21\}$	21	7745	841/18588	0.1023
22	$\{1, \dots, 22\}$	22	8464	8427/194672	0.1010
23	$\{1, \dots, 23\}$	23	9217	7264/175123	0.1083
24	$\{1, \dots, 24\}$	24	10001	11554/290029	0.1020
25	$\{1, \dots, 25\}$	25	10816	1657/43264	0.1098
26	$\{1, \dots, 26\}$	26	11664	3443/93312	0.1019
27	$\{1, \dots, 27\}$	27	12545	10268/288535	0.1004
28	$\{1, \dots, 28\}$	28	13457	2312/67285	0.1025
29	swapped set	55	14400	3349/100800	0.0985
30	swapped set	60	15377	989/30754	0.0961
31	swapped set	64	16384	5869/188416	0.1036
32	swapped set	60	17424	11579/383328	0.1054
33	swapped set	60	18497	16814/573407	0.1029
34	swapped set	60	19601	4467/156808	0.1036
35	swapped set	72	20737	342/12349	0.1082
36	swapped set	72	21904	367/13616	0.1008
37	swapped set	76	23105	32142/1224565	0.0986
38	swapped set	75	24337	74066/2896103	0.1017

For $n \leq 28$ the velocities are $\{1, \dots, n\}$. For $n \geq 29$ they are *not* $[n]$: the search swaps velocities (all $\leq 2n + 2$) and most of those sets drop large primes and add smooth numbers; all velocities are distinct positive integers and the trivial repeated-velocity cover is never used (both producer checkers and the auditor's checker reject repeated velocities).

Margins. The exact margins $1/\gamma(v; s) - (n+1)$ lie in [0.0961, 0.1098] for $n = 18, \dots, 38$ (minimum at $n = 30$, maximum at $n = 25$; `AUDIT.md` §2a, correction 6), so $E_n \geq 0.096$ for these n. (The value 0.125 that appears in the producer's text is the search *target* radius $\delta^* = \frac{1}{n+1} - \frac{1}{8(n+1)^2}$, not an achieved margin.) The margins are too small to chain from one n to the next, and no uniform rule was found; each n was searched separately.

Where the conjecture is known to fail. With the cited literature ([L]): $n = 5, \ldots, 17$ [1] (Prop. 5.14, as cited by the producer); $n = 18, \ldots, 38$ this paper (to our knowledge the first published certificates; see Section 7); $n \geq 95$ [3]. $n = 39, \ldots, 94$ **remains open.**

What was searched for $n \geq 39$. The search stopped because the total CPU budget (29.75 core-hours of the 30-core-hour cap) was reached. Attempts per n at the frontier (producer's driver logs, `data/RESULT.md`): $n = 39$: 0/23 attempts (6900 CPU-s); $n = 40$: 0/15 (4170 CPU-s); $n = 41, \ldots, 59$: 0/12 each (3600 CPU-s each, i.e. 12 attempts of 300 s); $n = 60$: 0/11 (3301 CPU-s); $n = 61, \ldots, 94$ **were never attempted.** For comparison the successes at the frontier took 1/20 attempts at $n = 36$, 1/8 at $n = 37$, 1/15 at $n = 38$. Failure to find a certificate for $n \geq 39$ says nothing about whether certificates exist.

3 Method

Producer code is in `data/code/` (Python and C; the C sources are also stored as `.c.txt` because the bundler keeps only listed suffixes; compiled binaries are not shipped).

Search (untrusted, Tier [X]). Floating-point coordinate moves on the uncovered measure at a fixed radius δ^*: for runner i the set not covered by the others is folded mod $\frac{1}{v_i}$ and the best window position is found exactly among the breakpoints (`cover_ls.py`, ported to C as `cover_ls_fast.c` with velocity swaps, restarts and an init file). Earlier calibration tools (chain-LP min-max descent, continuation in n, a smooth surrogate, a deletion probe) reached the published optima at $n = 5, 6, 8$ but failed at $n = 9, 10, 12$ and could not get below $\frac{1}{n+1}$ for $12 \leq n \leq 30$; they are kept in `data/code/` as superseded. The C search was rebuilt several times during the run, so certificates $27, \ldots, 38$ come from different binary versions; this does not affect validity, since only the checkers are trusted. Each float solution is rounded to $s_i = a_i/D$ with $D \geq 2/(\frac{1}{n+1} - \delta^*)$ and is then passed to the checkers. What worked at the frontier was repeated short restarts (for example $n = 36$ failed 17 times at 200 s and then succeeded in the third attempt of a 300-s batch); chain-LP polishing, a gap-repair LP, larger velocity pools and slack changes did not.

Producer's checkers (Tier [C]; same author). *Checker 1* (`slr_exact.py`, Python integers): every arc endpoint $(k - s_i \mp \delta)/v_i$ is scaled by $M = \mathrm{lcm}(v)\, D\, (n+1)$ to an integer and a greedy sweep decides whether the open integer intervals cover $[0, M]$. *Checker 2* (`check_cover.c`, `__int128`): the open arcs cover the circle iff every right endpoint of every arc lies strictly inside another arc, tested by cross-multiplication per runner pair; it parses the JSON itself. The two use different criteria and different languages, but **they were written by the same agent and are not independent in authorship**. All certificates were re-checked from the stored JSON alone by `verify_all.py` (`data/logs/verify_all.json`).

The auditor's third checker (Tier [C], independent of the producer). Written from scratch by the auditor, who read none of the producer's checkers (`data/code_auditor/audit_check.py`, standard library only). *Method A*: exact interval subtraction with `fractions.Fraction`; start from the closed set $[0, 1]$ and remove every integer translate of every open arc; the arcs cover if nothing is left, single points count as uncovered. *Method B*: the exact value $\gamma(v; s)$ in integer arithmetic, evaluating $f(t) = \min_i \|s_i + v_i t\|$ at every candidate maximiser (all pairwise crossings $s_i + v_i t = \pm(s_j + v_j t) \bmod 1$ and all single-runner peaks); the arcs cover iff $\gamma < \delta$. Both methods say COVERED for all 34 certificates. We re-ran this checker on all 34 files: 34/34 valid, both methods COVERED, and the printed γ values equal the auditor's table (`data/logs/auditor_checker_rerun.jsonl`). As a literature consistency check, for $n = 5, \ldots, 15$ the certificates use $v = [n]$ and each certificate's $\gamma(v; s)$ is at least the exact $\gamma_{\mathrm{sh}}([n])$ of BCS Table 2 (all 11 are; e.g. $n = 5$: $189/1154 \geq 15/94$).

4 Lean (Tier [A] pending a statement lock)

The statement. The auditor's generator `gen_lean.py` (untrusted; deterministic) writes, for each certificate, a file importing `Mathlib.Tactic` only (never `import Mathlib`); for $n = 18$:

```
def v : Fin 18 → ℕ := ![1, 2, ..., 18]
def s : Fin 18 → ℚ := ![3521/5777, ..., 4869/5777]   -- a_i / D of cert_n018.json
theorem v_injective : Function.Injective v := by decide
theorem v_pos : ∀ i, 0 < v i := by decide
theorem cover : ∀ t : ℝ, ∃ i : Fin 18, ∃ k : ℤ,
    |((s i : ℚ) : ℝ) + ((v i : ℕ) : ℝ) * t - (k : ℝ)| < 1 / 19
```

`cover` says that for every real t some runner is within distance $< \frac{1}{n+1}$ of an integer, which is exactly the negation of the conclusion of Conjecture 1.1(ii) for this (v, s); together with `v_injective` and `v_pos` (distinct positive velocities) it is a counterexample of size n. It needs neither the maximum γ nor compactness. The proof reduces t to $t - \lfloor t \rfloor \in [0, 1)$, splits on a chain of arc right endpoints (82 cases at $n = 18$, 449 at $n = 38$) computed by the generator in exact rationals, and closes each case with `linarith`; the kernel re-checks every case, so the generator is not trusted.

What the auditor ran (`AUDIT.md` §2c; the auditor's run, not ours). All 34 files $n = 5, \ldots, 38$ compiled with `scripts/certified_numerics/lean_pinned.py` (Lean 4.34.1, Mathlib `d13f23b7`) with **rc 0**, no banned token (`sorry`, `admit`, `native_decide`, `axiom`); `cover` and `v_injective` depend on {`propext`, `Classical.choice`, `Quot.sound`} and `v_pos` on `propext` only (`data/lean/lean_all_summary_auditor.json`; 3456 s in total on one core). Controls: the positive control (n=5, $\delta = 1/6$) has rc 0 with standard axioms; the same file with δ tampered to the exact $\gamma = 15/94$ has rc 1 (`linarith` fails), as it should. An early $n = 38$ attempt with a parse error still printed an axioms line, so the return code and not the axiom line is the gate.

Our re-run (sample $n = 18$ and $n = 38$) (`ledger/SLR-A-0001`, `data/logs/lean_gate/`; script `data/code/lean_gate_sample.py`). For each file we ran, on one core of a loaded machine: (1) `lean_pinned.compile_file` with an olean (Lean 4.34.1, Mathlib `d13f23b7`): $n = 18$ rc 0 in 60.7 s, $n = 38$ rc 0 in 347.3 s; (2) the repository's `anse/formal/lean_runner.py verify_file` gate with `lean_cmd` pointed at the pinned toolchain (the way the producer of the sibling E1 result ran it): for both files all three theorems are compiled, not `sorryAx`, with only whitelisted axioms (`cover` and `v_injective`: `propext`, `Classical.choice`, `Quot.sound`; `v_pos`: `propext`); (3) `leanchecker` on the olean (`Programmes.SLR18.SLRN18`, `...SLRN38`): rc 0 for both. Controls through the same three gates: `SLRP1.lean` rc 0 with standard axioms; `SLRP1neg.lean` (the tampered $\delta = 15/94$) rc 1 at compile, `lean_runner` reports it as failed, and `leanchecker` finds no olean (rc 1). So the file-mode gate of `lean_runner.py` *can* be run on these files, but we ran it on the sample only (2 of 34 plus the control).

Generator, hashes, and the link to the certificates. `formal_cert/Programmes/SLR18/` holds the generator (`auditor_tools/gen_lean.py`) and the 34 generated files `SLRN05.lean` ... `SLRN38.lean` plus the controls `SLRP1.lean` and `SLRP1neg.lean` (the latter fails by design), placed there as the audit's correction 8 asks (uncommitted at writing). The generator is deterministic: regenerating $n = 18$ and $n = 38$ gives byte-identical files with sha256 `19a312563d9277568341a0a01bb21c6f83c4c8fced45d85869fa7fd379faaa65` for `SLRN18.lean` and `9dcac924e547ad82f39f7bd77ed4f8bfc70cccf44cee39f4d4136a3f01154dd4` for `SLRN38.lean`, the hashes the auditor reported. A script of ours (`ledger/check_lean_literals.py`, `data/logs/lean_literal_check.json`) checks for all 34 files that the typed literals v and s equal the certificate JSON, that the `cover` statement is the template above with only n substituted, that the only import is `Mathlib.Tactic` and that no banned token occurs: all 34 pass.

Why this is not yet plain Tier A. The binding repository rule is that Lean verification goes through `anse/formal/lean_runner.py` and a statement lock. The auditor used `lean_pinned.py` plus a manual `#print axioms`, as its brief asked, not `lean_runner.py`, and no statement lock was run. We ran the file-mode gate of `lean_runner.py` on the sample above only; we created no statement-lock entry (the repository's other lanes hash statements with LeanMaster's `tools/statement_lock.py` in a scratch root, but it is not clear to us that this is the established way for programme files, and we did not guess) and **the statement lock is owed**. In the ledger the Lean rows are therefore Tier [A] with an `ai_audit` field and the gate field `audit` left null, as in the sibling papers, and the gate flags them. No person has certified that the Lean statements mean what their gloss says.

5 Controls

Producer's controls (run before any $n \geq 18$ was counted; `data/logs/controls_run1.json`, `data/logs/fuzz_agree.json`).

id	control and outcome
P1	γ exact of (1..5) with shifts $(0, 46, 38, 47, 72)/94$ is 15/94 (argmax $t = 45/94$, BCS Prop. 5.11); both checkers COVERED at $\delta = \frac{1}{6}$
P2	$(2, 3, 4, 5, 6, 8)$ with $(0, 29, 17, 0, 16, 22)/30$: $\gamma = 2/15$; both COVERED at $\frac{1}{7}$
P3	own shifts for $[n]$, $n = 5..17$: all 13 certified by both checkers
N0	P1 shifts at $\delta = \gamma = 15/94$ (tangency): both checkers reject
N1	$[n]$, $s = 0$, $\delta = \frac{1}{n+1}$, $n = 2..94$: 0 of 93 certified; $\gamma = \frac{1}{n+1}$ checked exactly for $n \leq 12$
N2	single-shift perturbation by $\pm 1/(10 \sum v)$: at the published δ ($P1$ at $\frac{1}{6}$, $P2$ at $\frac{1}{7}$) 0 **of** 10 **and** 0 **of** 12 **broke** (the margin exceeds the perturbation; reported, not hidden; the preregistered wording "the covering must break" is false there); at near-tight δ 4/10 and 5/12 broke; on new certificates $n = 18$: 10/36, $n = 30$: 4/60, $n = 35$: 5/70, $n = 37$: 5/74, $n = 38$: 6/76 broke; checker disagreements 0
N3	$s = 0$, up to 14 distinct speeds (3000 random sets and 5 known tight sets), $\delta = \frac{1}{k+1}$: 0 certified (consistent with the lonely runner conjecture for at most 14 speeds, Allikvere [4], [L])
F	agreement fuzz, 600 random small instances $\times 4$ deltas (γ exactly, $\gamma \pm 10^{-6}$, random): 2400 tests, 0 mismatches among checker 1, checker 2 and $\gamma_{\text{exact}} < \delta$

Auditor's adversarial tests (`data/code_auditor/adversarial.py`; `AUDIT.md` §2b): $[n]$ with $s = 0$ and $\delta = \frac{1}{n+1}$ for $n = 2..40$: 0/39 covered, $\gamma = \frac{1}{n+1}$ exactly (Dirichlet); the published values 15/94, 2/15 and, for (1..6) with $s = (0, 39, 37, 51, 62, 54)/67$, 9/67 reproduced exactly; tangency at $n = 5, 18, 29, 38$ not covered at $\delta = \gamma$, covered at $\gamma + 10^{-12}$, not covered at $\gamma - 10^{-12}$; a duplicated velocity, a zero velocity and $\delta = \frac{1}{n}$ each flagged invalid; the trivial repeated-velocity cover ($v = 1^6$, $s = i/6$) covers but is flagged invalid; shifting one runner by $\delta/2$ in either direction broke 36/36 perturbations at $n = 18$ and 72/76 at $n = 38$ (four runners redundant) with 0 disagreements between methods A and B; a fuzz of 400 random small instances $\times 4$ deltas had 0 mismatches, and a 4000-point float grid never exceeded the exact γ.

6 Tiers, and what is not claimed

statement	tier	basis
Definition of the shifted conjecture; strictness $\Leftrightarrow$ open arcs cover; prior results ($n \leq 17$, $n \geq 95$, $n \leq 4$ proved)	[L]	BCS, Zhang, Poliakova, as read by the auditor
For $n = 5..38$ the exact covering statement and distinctness/positivity of v	[A] pending lock	auditor's Lean replay (34/34); our re-run for $n = 18, 38$
Each certificate is covered; exact $\gamma(v; s) < \frac{1}{n+1}$	[C]	three exact checkers (two same-author, one independent)
$E_n \geq 0.096$ for $n = 18..38$	[C]	auditor's exact γ (method B)
The search that found the certificates; the gaps and stall diagnostics	[X]	floating point, untrusted
Failure for $n = 39..94$	—	**open**; no certificate within the budget is not evidence either way

What is not claimed. (i) No breakthrough: this continues a disproof that already exists for $n \leq 17$ and $n \geq 95$. (ii) **Nothing about the unshifted lonely runner conjecture**, which is a different statement ($s = 0$). (iii) Upper bounds $\Gamma(n) < \frac{1}{n+1}$ only. No exact γ_{sh} values. The auditor's exact $\gamma(v; s)$ gives the explicit lower bounds $E_n \geq 1/\gamma(v; s) - (n + 1) \geq 0.096$ for $n = 18..38$ (correction 7). (iv) **We do not claim the shifted conjecture fails for all** $n \geq 5$: the preregistered title said so, but the result does not support it, because $39 \leq n \leq 94$ is open. (v) The Lean files check the covering statement for the listed (v, s); they do not formalise γ, the conjecture, or the literature.

7 Literature position and scoop check

Dated 2026-10-28 (producer) and 2026-10-31 (auditor).

- **alphaXiv** (recency-ranked discovery, and with `published_after` 2026-10-01 / 10-20 / 10-24) for "shifted lonely runner", "lonely runner" and "counterexamples": the only shifted-conjecture counterexample papers are arXiv:2603.24784 and 2609.23952. Newer hits concern other problems (for example 2610.23790 on the unshifted $ML(V)$, 2610.37711 view-obstruction for spheres, 2610.35206 a runner-flow PDE); the producer read titles and abstracts only.
- **Web search**: no source settles any n in 18..94. Zhang v3 (18 Oct 2026) and Poliakova v1 both call the range open; BCS v4 (24 Oct 2026) states failure only for $n = 5..17$ and $E_n \geq 1$ only for $n \geq 95$.
- **No conflict found.** The shifted conjecture is proved only for $n \leq 4$ (Alcántara–Criado–Santos; BCS Remark 1.2). Nothing published gives a lower bound on γ_{sh} or $\Gamma(n)$ for $18 \leq n \leq 38$. Beck–Everett (arXiv:2609.06259 v2, Thm 3.2) require a shifted counterexample to satisfy a small relation $m \cdot v = 0$; the certificates do ($[n]$ has $2 \cdot 1 - 2 = 0$ and the swapped sets for $n \geq 29$ contain many relations $a + b = c$). Zhang's Lemma A.1 is consistent with them. BCS Table 2 gives $1/\gamma_{\mathrm{sh}}(1..15) = 18$ ($E = 2$), so it is plausible that $[n]$ still fails at $n = 18..28$.
- **Criado's repository** (latest commit 2026-04-24) has outputs only for $n \leq 13$ (`consecutive.out`) and $n = 5, 6, 7$ censuses; only the listing and that file were looked at, at the end, as part of the scoop check, and nothing from it was used as a seed.

Residual risk. Not checked: arXiv listings after the alphaXiv index date, unpublished BCS/Criado computations (the producer saw repository outputs only up to $n = 13$), and private communications. "To our knowledge, the first published certificates for $18 \leq n \leq 38$" is not a priority claim.

A courtesy note. We have not contacted the authors of the cited preprints (Blanco–Criado–Santos, Zhang, Poliakova), and they have not seen this work. Before any wide announcement the owner may wish to inform them.

8 Audit corrections applied

The independent audit (`AUDIT.md`, §5) required eight corrections to the producer's report; they are applied in this paper as follows. The producer's `data/RESULT.md` and `data/result.json` are the pre-audit text (their hashes are recorded in `result.json`, so they were not edited); this paper supersedes them.

1. **Claim wording on checkers.** The producer wrote "two independent checkers". Here: exact integer arithmetic by two producer checkers (different criteria and languages, same author) and by an independent auditor checker; the covering statement kernel-checked in Lean by the auditor (Section 4).
2. **Tier wording.** Not "C, no Lean replay" but: A by the audit brief's criterion for $n = 5..38$ (auditor's Lean kernel replay of the exact covering statement), repository gates (`lean_runner.py` and statement lock) pending, on top of three exact checkers (one independent). Hence "Tier [A] pending a statement lock".
3. **Status line.** Counterexamples are certified for every $n \in [18, 38]$. With $n = 5..17$ (BCS) and $n \geq 95$ (Poliakova) the shifted conjecture is known to fail for $n \in 5..38$ and $n \geq 95$; $n \in 39..94$ stays open.
4. **Lean attribution.** The producer did no Lean replay; the replay is the auditor's. Standard axioms; Tier [A] by the audit brief's criterion, repository gates pending.
5. **Hash coverage.** The 333 files of `logs/fast_tmp/*.txt` are raw search scratch, not in the hash list of `result.json`. We omit them from this paper's `data/` and from the bundle.
6. **Margin range.** "The margins are $E \approx 0.125$" is replaced by the exact $1/\gamma(v; s) - (n+1) \in [0.0961, 0.1098]$; 0.125 is the search target radius.
7. **Bound.** "No value of E_n beyond $E_n > 0$" is replaced by $E_n \geq 1/\gamma(v; s) - (n + 1) \geq 0.096$ for $n = 18..38$; still no exact γ_{sh} values.

8. **Commit generator and Lean.** `gen_lean.py` and the generated `.lean` files are placed (uncommitted) under `formal_cert/Programmes/SLR18/`; they are re-run through the pinned toolchain here, and through `lean_runner.py` for the sample (Section 4); a statement lock is owed.

Two further points on the producer's text. The producer's note that frontier attempts covered 39..59 with 12 attempts of 300 s each is refined above from the driver logs ($n = 39$ and 40 had more attempts; $n = 60$ also had 11; 61..94 never). Everything in `data/RESULT.md` about "independent checkers" and "Tier C, not A" is superseded by items 1, 2 and 4.

9 Claim ledger

`ledger/build_ledger.py` files 13 claims: 3 at Tier [A] (`SLR-A-0001`: the sample $n = 18, 38$ through all three gates; `-0002`: all 34, the auditor's run; `-0003`: Lean controls), 2 at [L] (definition; prior results), 5 at [C] (`SLR-C-0001..0005`: producer's checkers, auditor's checker, controls, Lean-literal check, and the headline `SLR-C-0005`) and 3 at [X] (search effort and absences; scoop check; what was not run). Each carries a content-addressed evidence blob in `ledger/evidence/`; each Tier [A] claim also carries an `ai_audit` field. The headline is filed at Tier [C] because its evidence kind (an argument assembling the rows above) is capped at [C] by the gate. The gate field `audit` is left null: the audit was done by an AI agent, not a person, so `LEDGER_UNAUDITED_TIER_A` stays open. The gate (`tools/ledger.py ledger.json --evidence-dir evidence`; verbatim in `ledger/gate_output.txt`) exits with code 1: 0 blocking findings and 3 flags, all `LEDGER_UNAUDITED_TIER_A`, one per Tier [A] row (`SLR-A-0001..0003`). That is the honest state.

10 Limitations

- **AI audit only; human audit and statement lock owed.** The audit is by a separate AI agent instance, not a person. No person has certified that the Lean statements mean what their gloss says; no statement lock exists; the repository's `lean_runner.py` gate was run on a sample ($n = 18, 38$) only, and the other 32 files rest on the auditor's run with `lean_pinned.py`.
- **Still unverified (audit §6).** (a) The repository-gate Lean verification of all files; (b) scoop completeness beyond the alphaXiv index and web search, including unpublished BCS computations; (c) the per-log CPU figures (101746.4 s by log plus 5344.8 s manual runs = 107091.2 s = 29.75 core-hours; the controls/checks entry of 300 s is an estimate; the margin to the 30 core-hour cap is about 900 s); (d) the producer's search code, which is untrusted and not needed for validity; (e) a full reading of every log for third-party text (grep-level only: no copy of Criado's `consecutive.out` and no paper text is tracked).
- **The two producer checkers share an author.** Their agreement is not independent evidence; the independent evidence is the auditor's checker (two methods) and the Lean replay.
- **Literature premises are taken as stated** ([L]): the definition and the statements about $n \leq 17$ and $n \geq 95$ were read by the auditor on alphaXiv, not re-derived; $n = 5..17$ are cited from BCS and used here only as positive controls.
- **Scope.** $18 \leq n \leq 38$ only. The search is heuristic. $39 \leq n \leq 94$ is open; $61 \leq n \leq 94$ was never attempted. Nothing here concerns the unshifted conjecture, exact γ_{sh} values, or any physical claim.
- **Bundle contents.** The 333 raw search scratch files `logs/fast_tmp/*.txt` are omitted (correction 5); compiled binaries are not shipped (C sources as `.c.txt`); the Lean files are the auditor's generated files, copied unchanged.
- **Deviations from the preregistration** (from the producer's note): velocities for $n \geq 29$ are swapped sets, which the preregistration allows as "modified velocity sets"; the Poliakova uniform-rule route and add-a-runner chaining were not used; the N2 control wording was applied per runner and both signs (see Section 5); everything ran on one core, one process at a time.

11 Reproducibility

Repository AutoevolveAI, branch `worktree-openai-math-discovery`; HEAD when this paper was written `1e29c538f2ceacd0e644fc990cfba32ce45fa0d7`; the producer's commit is `bbfc5653a4d72f0f7dc311d109095c3fc7dc567b`. The paper, the Lean files and the auditor's tools are uncommitted working-tree files at that time; nothing was pushed. Copies are in `data/`.

```
python3 -I data/code_auditor/audit_check.py data/certificates            # exact checker, methods A and B
python3 -I data/code_auditor/adversarial.py data/certificates            # edit the audit_check import path
python3 -I data/lean/gen_lean.py data/certificates/cert_n018.json N18 > SLRN18.lean
python3 scripts/certified_numerics/lean_pinned.py compile formal_cert/Programmes/SLR18/SLRN18.lean --timeout 3600
python3 -I ledger/check_lean_literals.py ; python3 -I ledger/gen_table.py
python3 -I ledger/build_ledger.py ; python3 -I ledger/run_gate.py
```

A sha256 of the generated Lean files

Hashes of the files in `formal_cert/Programmes/SLR18/` (the auditor's generated files, copied unchanged; computed by `ledger/gen_table.py`, which writes `table_lean_hashes.tex`; its content is pasted below so that every file name occurs in this source). SLRNnn is the certificate with $n = nn$; `SLRP1` is the positive control ($n = 5$, $\delta = \frac{1}{6}$), `SLRP1neg` the tampered control that must fail.

file	sha256
SLRN05.lean	15e6d53df2cf05e0632155e70bc2204fb33c5ca2a73113baec35267c70e7af03
SLRN06.lean	5c7e136f41566c094c6fc33dbf302fc610573c550d2512d330b3efb0cbc08c1e
SLRN07.lean	b92d6a11ca44193fe31c1b5797f238538a5996913fe1c0e2b74a2ca8cf408b6a
SLRN08.lean	015296176da2c35fe5fde50f95e00d2a861fce8f82b933248e9d8cde5b2086fb
SLRN09.lean	5badba94a2cdd9f4f34ae2464f94abf487a1b467bc9cb373dd0bfc2b81e68186
SLRN10.lean	c428c4cccb2dd43f6e57de466a8dfd42aa1115be82434041329bc135638c5c39
SLRN11.lean	c346028387e7f2973859056a4a57af82238809c3c17c90eea8f5d6d2b7c66b9a
SLRN12.lean	fa6d04a6785d4b722aaabf4b9683b06e6dcfbedc02dad0b16aa364ca7aa4b51a
SLRN13.lean	79513b6a09ee15633d8e1f9e0143c62256d872856874c489f8a15112cad42b6d
SLRN14.lean	d8ff1ded70568251739cddcff33bf34b6b053672be68d9ed3fb852dcc8763e0e
SLRN15.lean	4dd180bee0f7371f8bc2f3756b5cc757678f7fc7d01e7f798152edfd4692dee9
SLRN16.lean	d8b78fb8981803150e7fdf0a6816a23d1011e54a41cf20772b2f8b0833e14742
SLRN17.lean	43124a84b8d5fa3fda3595d2efa3ac9c11880afa29f621875f9faee7ed5d13fc
SLRN18.lean	19a312563d9277568341a0a01bb21c6f83c4c8fced45d85869fa7fd379faaa65
SLRN19.lean	caae6d38058b178b6fa7ebf943ed2d1dab15ad80e11a70639ccfb9e949dd0691
SLRN20.lean	a7a32baa86b00c9f4f11e0f6eeab0ded2ce5e88dab07b737c13fb63dbaf2ced8
SLRN21.lean	03be9fc9580fe7075fd272a4527e37117304d897c6e7d3073005e88e1f445e49
SLRN22.lean	d5415658e8ac91dfbd318ac7940ed542b00b1117e25a67a9b5755e8ff145b2ac
SLRN23.lean	66dfde9026c824a8af377f20517cd0b54d94a301b87775cd27c783e8f9896b73
SLRN24.lean	99fddb73a2fda9f4738c779f560fdaecd7342213cb49973495a8889d1d761f30
SLRN25.lean	8d925378f25370d192725252723e56c3a137dd8c3f8b5601e448dcb135fd2f5a
SLRN26.lean	be1f10e1eecb775e769b1da4034d37e43a472986c7c964fad6123ca051f6c740
SLRN27.lean	957d7ccee91230eb34ea4ab7f0e488fdd3b593f94f15389226dedd354fcdf087
SLRN28.lean	0e19c9a3c8c76e4795933c81f4d0aa45993524f2b2f3b27a1b3a066ddf3c1af7
SLRN29.lean	a4e7dae04f690b197be757ab95c114f2500e38b57c1c57e780265942ff290016
SLRN30.lean	9ce094b125828e6fc46f9d7abb3ec7ae4fb71c88cde73e3888829def2dddd8a0
SLRN31.lean	5b699517d072d30f50c24e9074bcedbab93365487dbc59d9c8fcb2ab08a8b16a
SLRN32.lean	3e3f8b64bbfc20d76acb84ba2c86405b44f4f120376bd6d43340009ea855ea58
SLRN33.lean	18a6250fd6947f5668f7de7b704b72393d0717f7aca4b17f069b2acf91606910
SLRN34.lean	0a52a0bddd662fd75b4fc032148359404db59e06e6465952135d27eef458b781
SLRN35.lean	64b91f789a8d4995944c48a50d4eda6e3afb9beb429b7a28fa480ea71022cfe1
SLRN36.lean	1225cb214684fba87e69e67858877f637502fe450fbc0cd43a12c1f062650145
SLRN37.lean	4d91758e24eb2ab410fa23548d064dd3cfbdfe1d6297dcfd154eb9080b495b81
SLRN38.lean	9dcac924e547ad82f39f7bd77ed4f8bfc70cccf44cee39f4d4136a3f01154dd4
SLRP1.lean	a4b28e1bb977754cbb7f6a18cef82f6633d9a94fbb9ff430ed2d0bc6bccb5d15
SLRP1neg.lean	82dad2898bf46e4e4788b3e43b8cdfbe036dc941b43bdc80e6d98f66e148343c

References

[1] Blanco, Criado, Santos, shifted lonely runner conjecture, arXiv:2603.24784 (v4, 2026-10-24). Conjecture 1.1(ii), Def. 1.3, Table 2, Props. 5.11, 5.12, 5.14, Remark 1.2 ([L]; title not reproduced here).

[2] Zhang, arXiv:2608.13599 (v3, 2026-10-18). App. A, Question 8.5, Lemma A.1 ([L]).

[3] Poliakova, arXiv:2609.23952 (v1). §1, Theorem 2, p. 3 ([L]).

[4] Allikvere, *Fourteen and fifteen lonely runners*, arXiv:2609.02604; cited only for the unshifted conjecture with at most 14 speeds (control N3, [L]).